

DEPLOYMENT

Claude Code on Microsoft Foundry

Learn about configuring Claude Code through Microsoft Foundry, including setup, configuration, and troubleshooting.

Deploying Claude Code across your organization? Talk to sales about enterprise plans, SSO, and centralized billing.

[View plans](#)[Contact sales →](#)

Prerequisites

Before configuring Claude Code with Microsoft Foundry, ensure you have:

- An Azure subscription with access to Microsoft Foundry
- RBAC permissions to create Microsoft Foundry resources and deployments
- Azure CLI installed and configured (optional - only needed if you don't have another mechanism for getting credentials)

❗ If you are deploying Claude Code to multiple users, [pin your model versions](#) before rolling out.

Setup

1. Provision Microsoft Foundry resource

First, create a Claude resource in Azure:

1. Navigate to the [Microsoft Foundry portal](#)
2. Create a new resource, noting your resource name
3. Create deployments for the Claude models:
 - Claude Opus
 - Claude Sonnet

- Claude Haiku

2. Configure Azure credentials

Claude Code supports two authentication methods for Microsoft Foundry. Choose the method that best fits your security requirements.

Option A: API key authentication

1. Navigate to your resource in the Microsoft Foundry portal
2. Go to the **Endpoints and keys** section
3. Copy **API Key**
4. Set the environment variable:

```
export ANTHROPIC_FOUNDRY_API_KEY=your-azure-api-key
```

Option B: Microsoft Entra ID authentication

When `ANTHROPIC_FOUNDRY_API_KEY` is not set, Claude Code automatically uses the Azure SDK **default credential chain**. This supports a variety of methods for authenticating local and remote workloads.

On local environments, you commonly may use the Azure CLI:

```
az login
```

ⓘ When using Microsoft Foundry, the `/logout` command is unavailable since authentication is handled through Azure credentials.

3. Configure Claude Code

Set the following environment variables to enable Microsoft Foundry:

```
# Enable Microsoft Foundry integration
export CLAUDE_CODE_USE_FOUNDRY=1

# Azure resource name (replace {resource} with your resource
name)
export ANTHROPIC_FOUNDRY_RESOURCE={resource}
# Or provide the full base URL:
# export
ANTHROPIC_FOUNDRY_BASE_URL=https://{resource}.services.ai.azure.c
om/anthropic
```

4. Pin model versions

⚠ Pin specific model versions for every deployment. Without pinning, model aliases such as `sonnet` and `opus` resolve to Claude Code's built-in default for Foundry, which can lag the newest release and may not yet be available in your account. Foundry has no startup model check, so requests fail when the default is unavailable. When you create Azure deployments, select a specific model version rather than “auto-update to latest.”

Set the model variables to match the deployment names you created in step 1.

Without `ANTHROPIC_DEFAULT_OPUS_MODEL`, the `opus` alias on Foundry resolves to Opus 4.6. Set it to the Opus 4.8 ID to use the latest model:

```
export ANTHROPIC_DEFAULT_OPUS_MODEL='claude-opus-4-8'
export ANTHROPIC_DEFAULT_SONNET_MODEL='claude-sonnet-4-6'
export ANTHROPIC_DEFAULT_HAIKU_MODEL='claude-haiku-4-5'
```

Background tasks such as session title generation use the small/fast model, normally a Haiku-class model. On Foundry, Claude Code defaults this to the primary model because not every account has a Haiku deployment. To use Haiku for background tasks, set `ANTHROPIC_DEFAULT_HAIKU_MODEL` to a Haiku deployment that is available in your account, as shown above.

For current and legacy model IDs, see [Models overview](#). See [Model configuration](#) for the full list of environment variables.

Prompt caching is enabled automatically. To request a 1-hour cache TTL instead of the 5-minute default, set the following variable; cache writes with a 1-hour TTL are billed at a higher rate:

```
export ENABLE_PROMPT_CACHING_1H=1
```

5. Run Claude Code

With the environment variables set, start Claude Code from your project directory:

```
claude
```

Claude Code reads `CLAUDE_CODE_USE_FOUNDRY` and the other Foundry variables from the environment and connects to your Azure resource on the first prompt. Unlike Bedrock and Vertex AI, Foundry has no interactive setup wizard, so the environment variables in steps 3 and 4 are the only configuration path.

Azure RBAC configuration

The `Azure AI User` and `Cognitive Services User` default roles include all required permissions for invoking Claude models.

For more restrictive permissions, create a custom role with the following:

```
{
  "permissions": [
    {
      "dataActions": [
        "Microsoft.CognitiveServices/accounts/providers/*"
      ]
    }
  ]
}
```

For details, see [Microsoft Foundry RBAC documentation](#).

Troubleshooting

If you receive an error “Failed to get token from azureADTokenProvider: ChainedTokenCredential authentication failed”:

- Configure Entra ID on the environment, or set `ANTHROPIC_FOUNDRY_API_KEY` .

Additional resources

- [Microsoft Foundry documentation](#)
- [Microsoft Foundry models](#)
- [Microsoft Foundry pricing](#)